



- ▶ Presentations due 12/1
 - ▶ Details on myCourses or in [Content > Term Project > 514 Term Project](#)
- ▶ Teams present in class 12/1 or 12/3
 - ▶ Teams sign up by contacting me on Slack (or I will pick randomly)
 - ▶ I will announce the date for presenting by Monday, 11/24
- ▶ Final Projects Due Mon, 12/8 by midnight
 - ▶ Includes user's guide
- ▶ Peer reviews due Mon, 12/8
 - ▶ Individuals that don't submit will lose points on project

Architecting for the Cloud: Avoiding Common Pitfalls

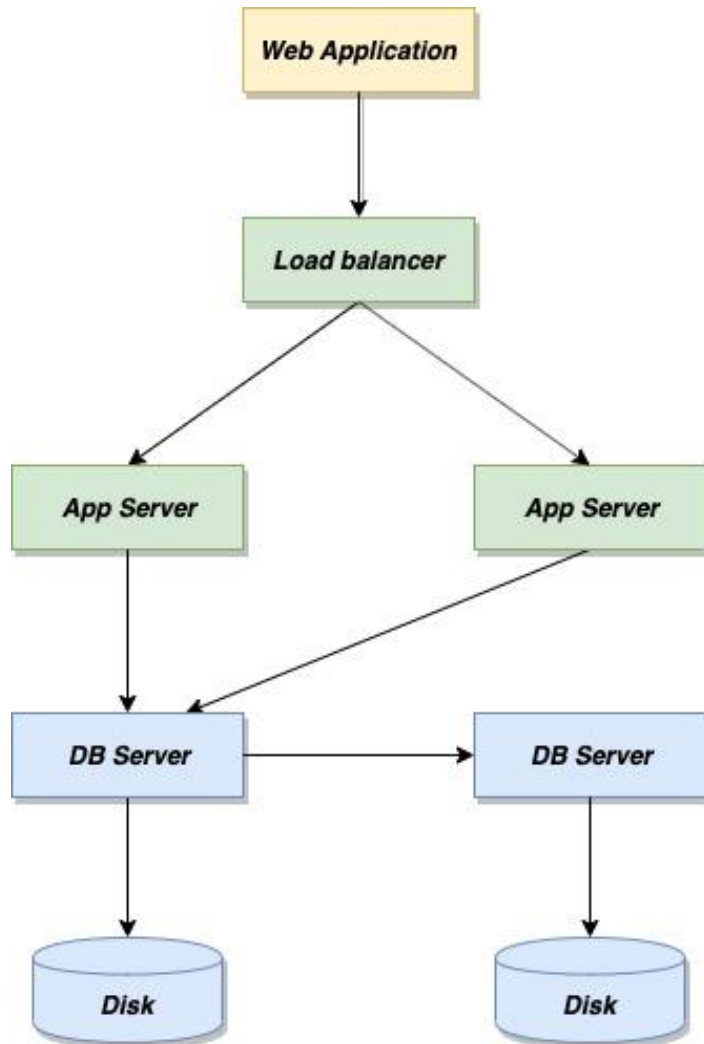
Engineering Cloud Software Systems

Department of Software Engineering
Rochester Institute of Technology



You are Presented with this Architecture to Build on the Cloud

3



- ▶ This is a brand-new web application your company is building for the first time
- ▶ Your company has no previous experience building in the cloud but understands the basics of AWS
- ▶ As a “cloud architect”, what are some of the mistakes you would want to avoid when you build this in the cloud?
 - ▶ Example, “Not knowing your cloud budget”

Top 10 Mistakes to Avoid

1. Not knowing your AWS infrastructure limits (way) ahead of time
2. Choosing the wrong AWS region
3. Not accurately predicting and controlling AWS cost
4. Not designing for failure
5. Not using Auto-Scaling
6. Avoiding Automation
7. Not using Infrastructure as Code (IAC)
8. Don't properly manage Identity, Access and Security
9. Don't do Continuous Integration Continuous Delivery/Deployment
10. Avoiding monitoring and metrics



Mistake #1: Not Knowing your AWS Infrastructure Limits (way) Ahead of Time

- ▶ The AWS infrastructure you provisioned on day-1 may eventually reach a limit and could crash
- ▶ Not knowing what that limit is and when will it be reached is a sure way to hit a wall
- ▶ How can you avoid this?
 - ▶ Execute load tests
 - ▶ You should know exactly what type of traffic your applications can handle with your current AWS infrastructure
 - ▶ You should know what the customer experience will be, and you should know at which point things will start to break
 - ▶ Capacity Planning
 - ▶ Based on load test results and your growth goals, you should know well in advance when it's time to beef up your AWS infrastructure
 - ▶ Don't wait until your customers are having a bad day using your product. **It will be too late!**



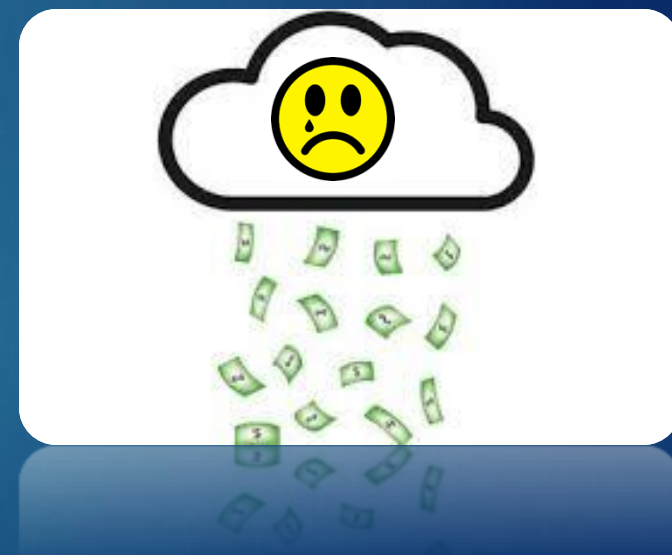
Mistake #2: Choosing the wrong AWS region

- ▶ A very common mistake is to choose an AWS region simply based on the location of your customers
- ▶ Not all AWS regions are created equal, there are substantial differences in price and feature availability
- ▶ If you think you'll use a certain AWS service in the near future, make sure it's available in your chosen region
- ▶ Some configurations can cost almost double, depending on the region you choose
 - ▶ Data transfer in Sao Paulo is 177% more expensive compared to N. Virginia
- ▶ Even within the US, some EC2 instances (like a t2.large) in N. California will cost you 30% more compared to N. Virginia



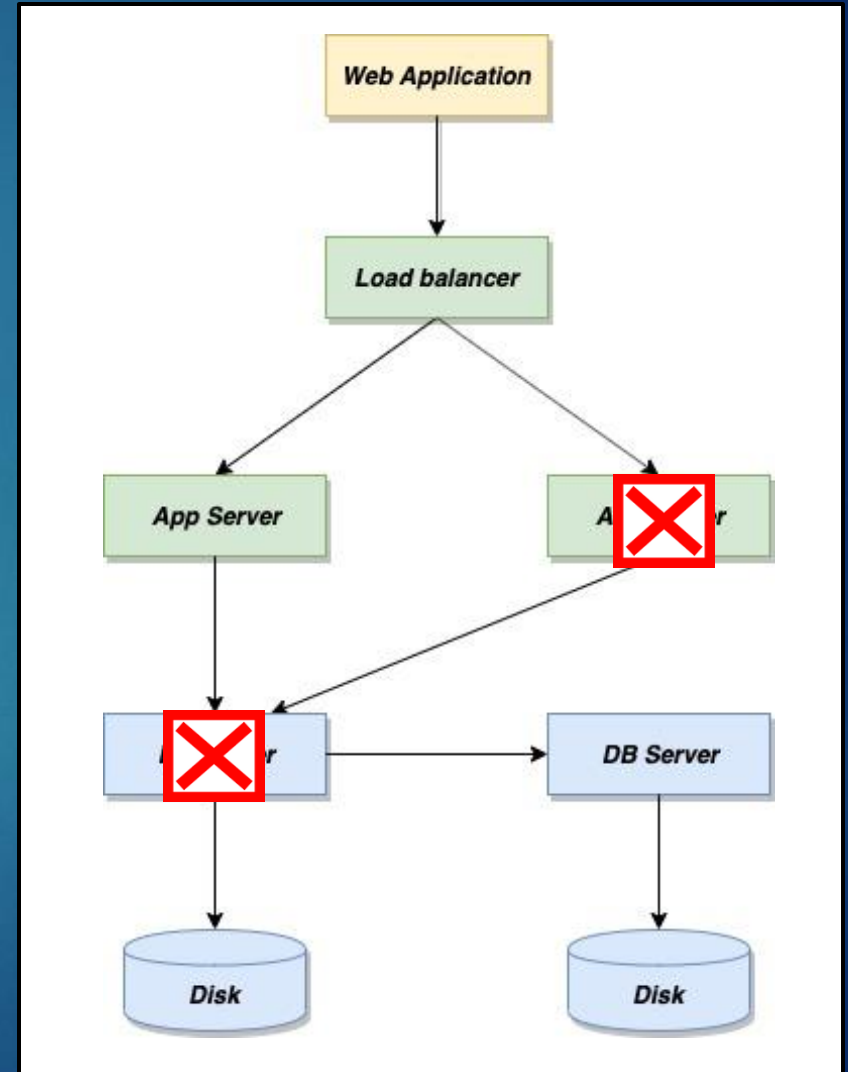
Mistake #3: Not Accurately Predicting and Controlling AWS Cost

- ▶ An application that costs more than it's worth will eat your profit
- ▶ For example, if you are paying \$2,000/month for cloud resources for an application that brings in \$1000/month, something is wrong
- ▶ Choosing the right instances for your AWS system is a fundamental decision one needs to confront
 - ▶ How many instances to choose?
 - ▶ What's the right size for an instance?
 - ▶ How to keep track of all the instances?
 - ▶ All these are basic yet vital decisions to be made
- ▶ AWS pricing is complicated, but not impossible to calculate. You need to know the type of resources your application consumes, their quantity and their corresponding AWS price dimension.
 - ▶ Example: data transfer (ingress/egress), ELB data processed, instance hours, EBS/S3 storage, billable API calls, Lambda executions and memory, etc. All this must be accounted for!



Mistake #4: Not Designing for Failure

- ▶ Imagine for the online web application that during the night, the hardware running your virtual machine fails
- ▶ Until the next morning when you go into work, your customers can no longer access your web application
- ▶ During the 8-hour downtime, your customers (who are now angry) search for an alternative and stop buying from you
- ▶ That's a disaster for any business



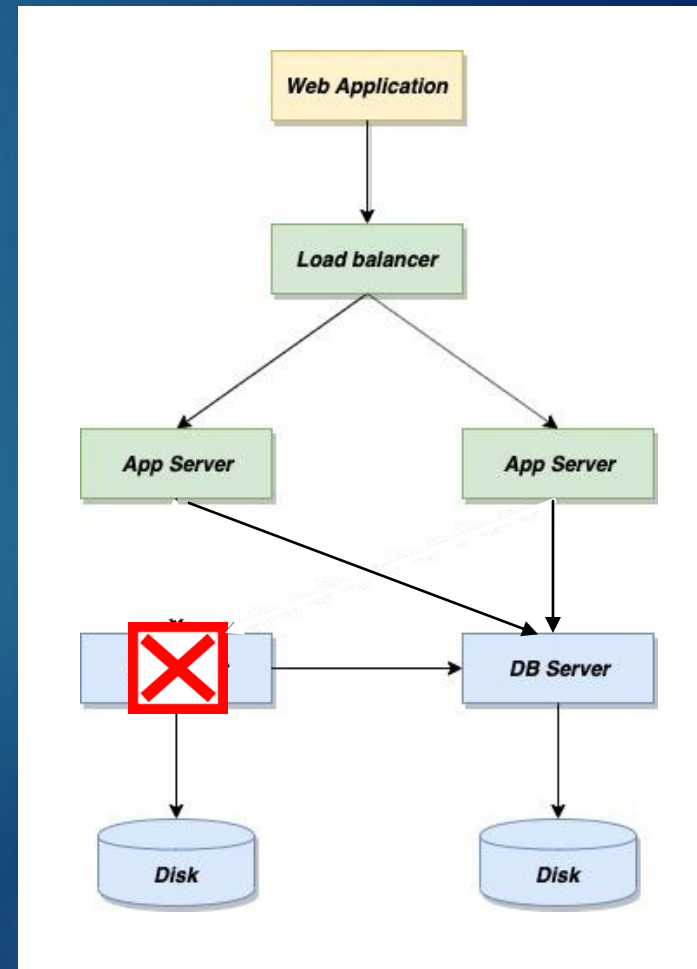
Mistake #4: Not Designing for Failure

- ▶ Virtual machines are not highly available by default
- ▶ The following scenarios could cause an outage of your virtual machine:
 - ▶ A software issue occurs on the host machine, causing the VM to crash (either the OS of the host machine crashes, or the virtualization layer does)
 - ▶ The computing, storage, or networking hardware of the physical host fails
 - ▶ Parts of the data center that the virtual machine depends on fail: network connectivity, the power supply, or the cooling system
- ▶ Remember: If the computing hardware of a physical host fails, all EC2 instances running on this host will fail



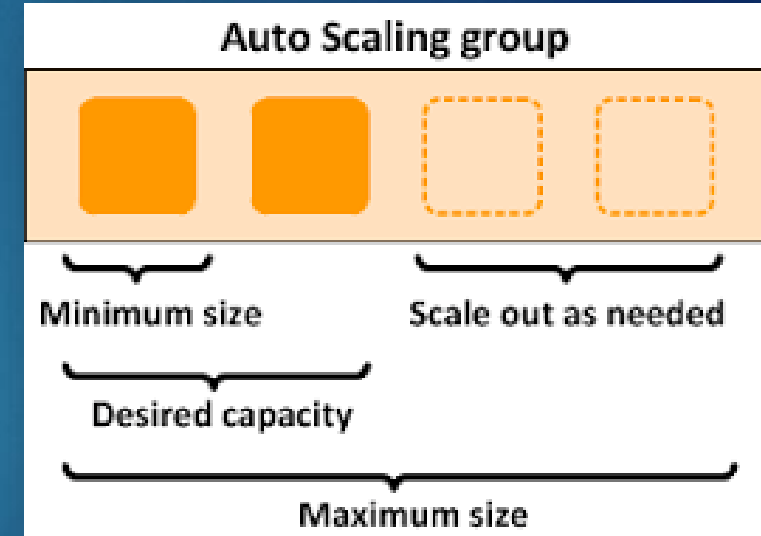
Mistake #4: Not Designing for Failure

- ▶ You don't want to over-engineer things at the beginning. But you also need to be prepared for failure. Here are two critical areas to consider:
 - ▶ Minimize likelihood of failure
 - ▶ Achieved by proper testing of your applications (e.g. failovers) as well as deployment steps, which can help to reduce human error
 - ▶ Using managed services such as RDS, Auto-Scaling or Elastic Load Balancer (ELB) can help to minimize the impact of these issues
 - ▶ Reduce time to recovery
 - ▶ You sure need to think about which tools and processes to put in place to make your applications recover as quickly as possible
 - ▶ These include: escalation, up-to-date documentation, automated recovery, updated runbooks, good logging, appropriate monitoring and alarming, ticketing systems
 - ▶ *"The best way to avoid failure is to fail constantly"*
 - ▶ If you aren't constantly testing your ability to succeed despite failure, then it isn't likely to work when it matters most — in the event of an unexpected outage (i.e. Chaos Engineering)



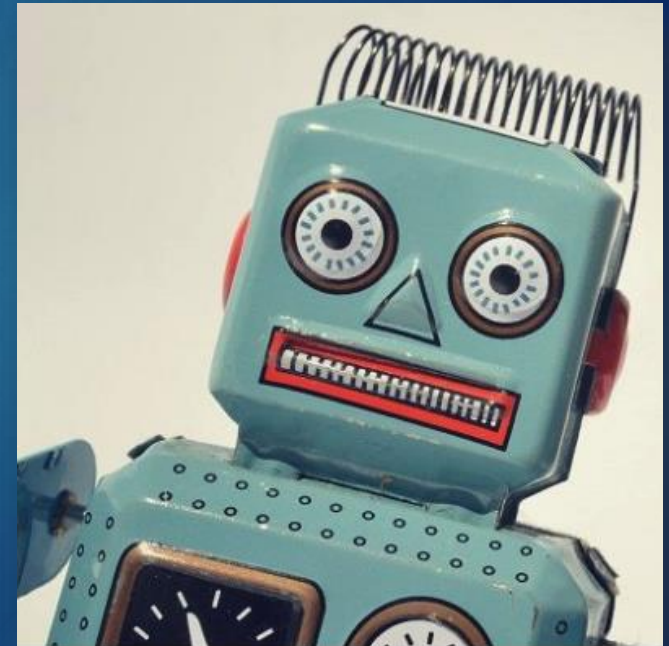
Mistake #5: Not using Auto-Scaling

- ▶ Every EC2 instance should be launched inside an **Auto-Scaling Group** even if it's a single EC2 instance
- ▶ The Auto-Scaling Group takes care of monitoring the EC2 instance, it acts as a logical group of virtual machines, and it's free*
- ▶ Auto-Scaling is achieved by setting alarms on metrics like CPU usage (of the logical group) or number of requests the load balancer received
 - ▶ If the alarm threshold is reached you can define an action like increase the number of machines in the Auto-Scaling Group
- ▶ The most common purpose for an Auto-Scaling groups is resiliency
 - ▶ Instances are put into a fixed-size Auto Scaling group so that if an instance fails, it is automatically replaced
 - ▶ Remember: The simplest use case is an Auto-Scaling group has a min size of 1 and a max of 1



Mistake #6: Avoiding Automation

- ▶ Not having automation in place is the best way to guarantee your team won't scale
- ▶ You don't need to have automation mechanisms for every single task, but you need to have some automation and gradually add tasks
- ▶ Don't wait until your team is overwhelmed with manual, tedious, time sucking activities
- ▶ Some suggestions:
 - ▶ Ask your team about the most painful manual tasks they execute. Measure by time spent and level of frustration.
 - ▶ Based on that, sort those tasks from most urgent to less urgent
 - ▶ Identify an automation task for each each manual task and estimate how much time it would take to implement
 - ▶ Prioritize automation for those tasks your team hates and where you'll get the biggest bang for your buck



Mistake #7: Not using Infrastructure as Code (IaC)

- ▶ IaC has a bit of a learning curve, and some templates can take time to build, but it's worth it
- ▶ Not using an IAC tool like Terraform means you'll have to manually create environments, and it also means you won't have a nice way to keep track of configuration changes
- ▶ Imagine you can create a big production-like environment in less than 10 minutes, run some load tests, shut it all down and launch it again the next day for more testing
 - ▶ Or rollback some configuration change that didn't work well in production, in minutes



Mistake #8: Don't Properly Manage Identity, Access and Security

- ▶ Your team will grow, you'll have more integrations with other systems, you'll use more API keys, you'll create more IAM Policies, Roles and Users, you'll set up more EC2 Security Groups, Network ACLs, you'll need to manage more encryption keys and certificates
- ▶ If not managed properly, all these things add up and turn very quickly into a dangerous mess
- ▶ Given security is the factor with potentially the most devastating consequences, you don't should not loose sight on security best practices from day-1 and as you grow



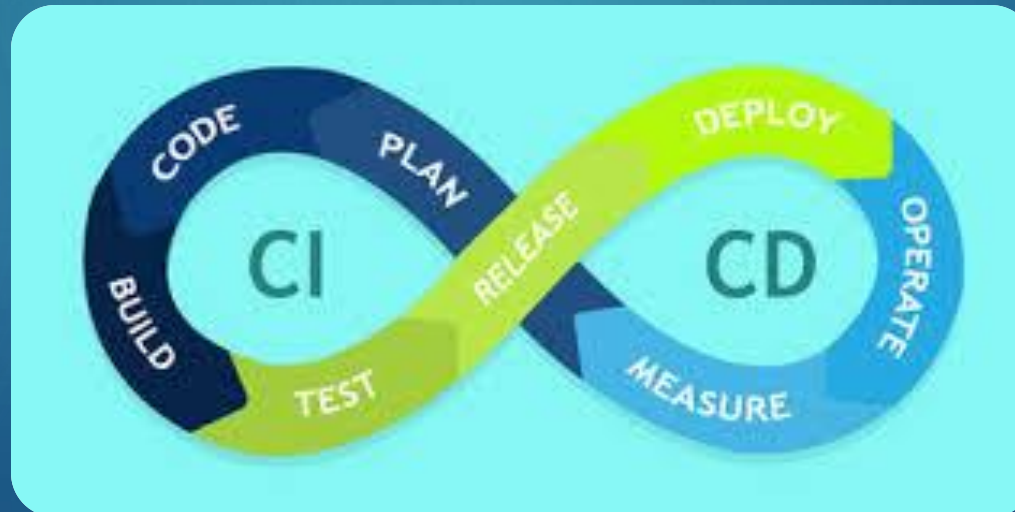
Mistake #8: Don't Properly Manage Identity, Access and Security

- ▶ Question: What is one of the most commonly seen and *dangerous* mistakes companies make when they use AWS services?
- ▶ Answer: Allowing sensitive data stored in S3 buckets to be publicly accessible
 - ▶ These buckets are secure by default but customers often loosen these access measures to enable development
 - ▶ In many cases, they are not re-assigned once development is complete and the service goes live
 - ▶ As a result, multiple, very public breaches of S3 security continually make the news
 - ▶ AWS administrators need to be sure to grant access to S3 (or *any* other AWS service) following the principle of “least privilege” making certain that only those properly trained and authorized to make changes within S3 have the access necessary to do so



Mistake #9: Don't do Continuous Integration Continuous Delivery/Deployment

- ▶ This is another way to guarantee your team won't scale
- ▶ You don't need to have the most sophisticated workflows in place from day-1, but you need to at least have a pipeline, which you'll gradually upgrade
- ▶ Once you start getting more feature requests, bugs, and your codebase grows, it will be more difficult to implement a good foundation

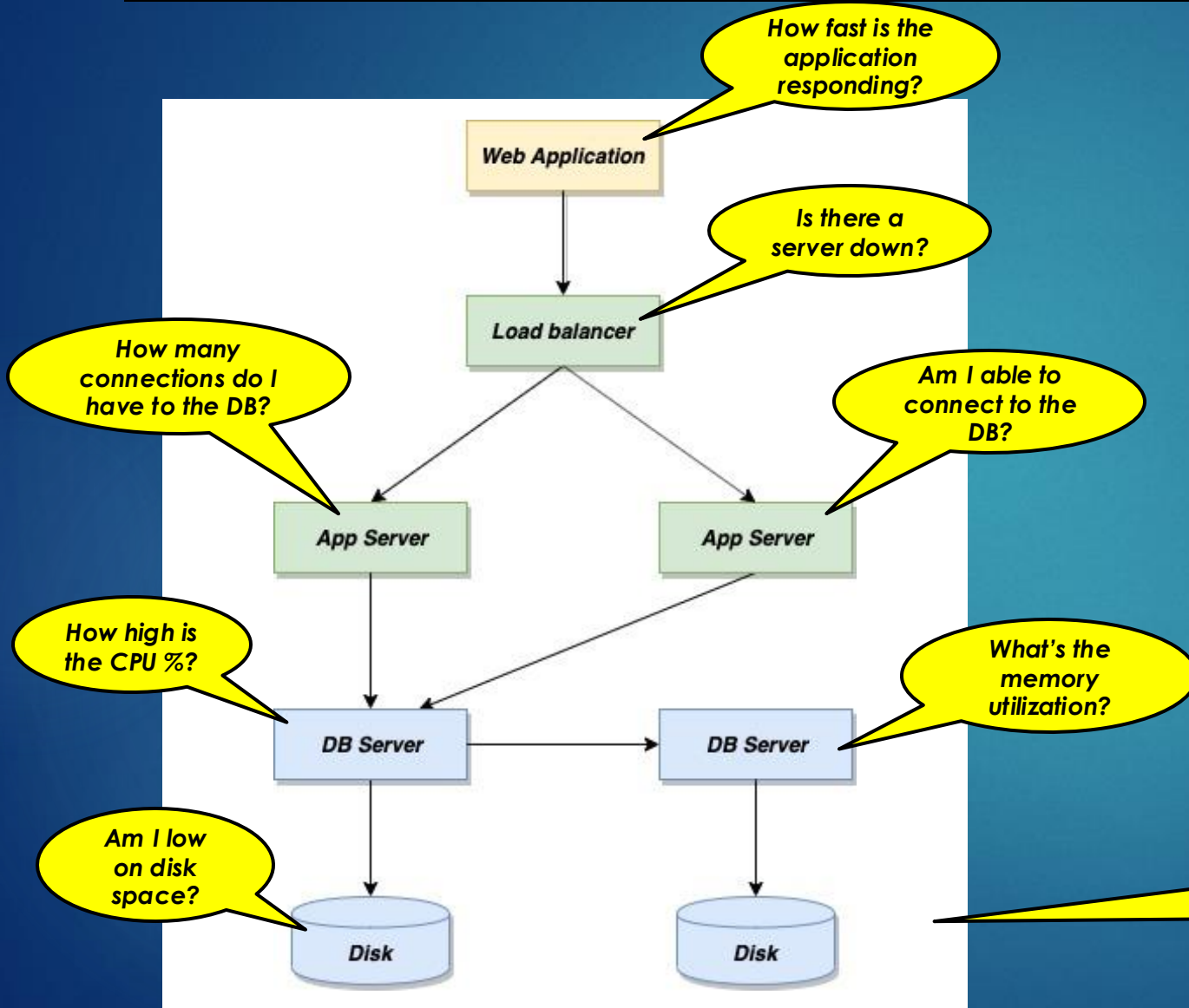


Mistake #10: Avoiding Monitoring and Metrics

- ▶ “You can’t manage what you can’t measure” is extremely important here
- ▶ Not having visibility on important metrics is a recipe for disaster
- ▶ Consider measuring for following categories:
 - ▶ **Business Metrics** - They tell you if you’re still making money (or not). They measure in real time your performance against business goals and objectives.
 - ▶ **Customer Experience Metrics** - They tell you what your customers see, if they’re having a good day using your application (or not). They reflect “symptoms” when things are not going well.
 - ▶ **System Metrics** - They tell you the root causes behind Customer Experience metrics. They also tell you if your systems are healthy, at risk, or already creating a bad experience for your customers.



Mistake #10: Avoiding Monitoring and Metrics



▶ Customer Experience Metrics

- ▶ Examples: response times, page load times, error 500s, etc.

▶ System Metrics

- ▶ Examples: CPU Utilization, memory utilization, disk I/O, queue length, etc.

▶ Business Metrics

- ▶ Examples: Revenue from application (e.g. orders submitted) vs. how much this all costs to run in the cloud

- ▶ Previously we would only alert on a few system statistics:
 - ▶ 90% CPU utilization for over 10 minutes
 - ▶ Free disk less than 10%
- ▶ Many times, when an alert was triggered, either the development team wasn't called, or when they were, it was already too late
- ▶ We decided to take proactive measures

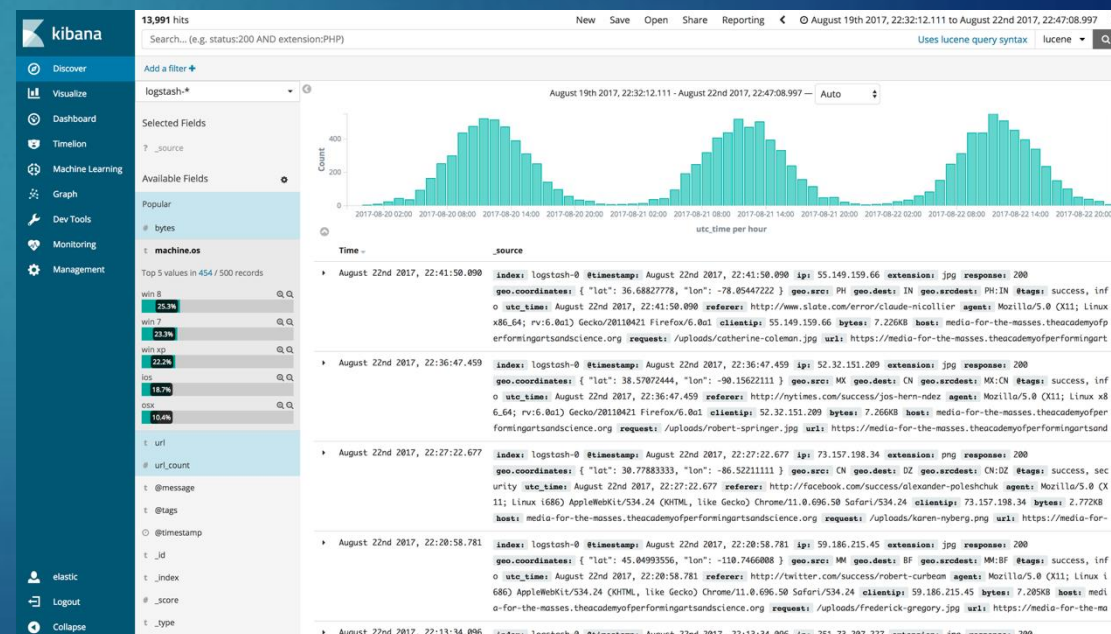


**BRACE
FOR
IMPACT**

Kibana

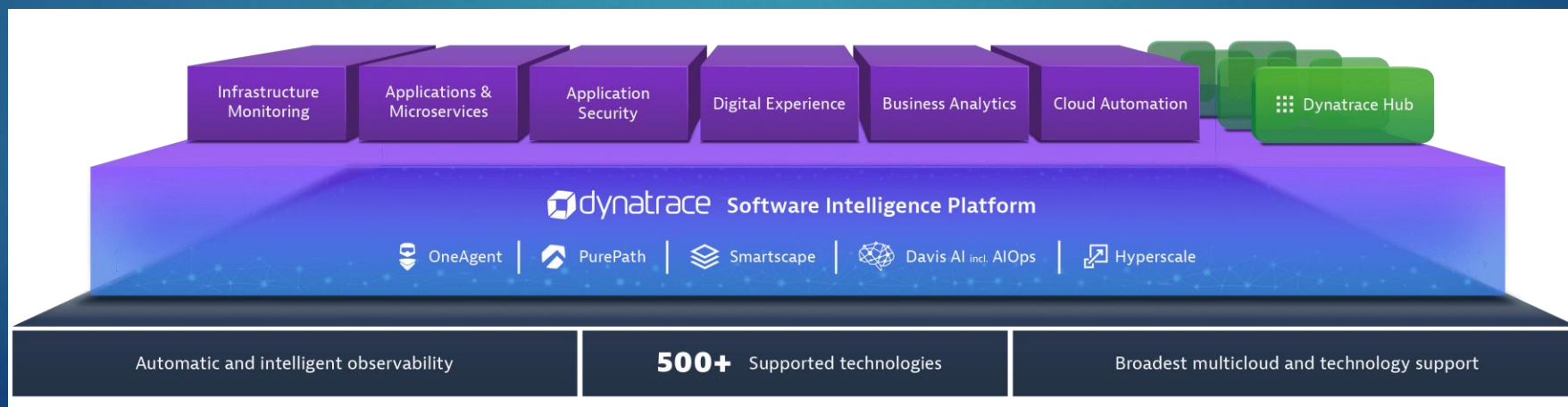
20

- ▶ Free and open-source front-end application that sits on top of the Elastic Stack, providing search and data visualization capabilities for data indexed in Elasticsearch
- ▶ All log files are automatically pulled into a Kibana dashboard, which makes troubleshooting much quicker
- ▶ Alerts can be created based on predefined conditions
 - ▶ Certain error messages are appearing in log files



DynaTrace – Game Changer

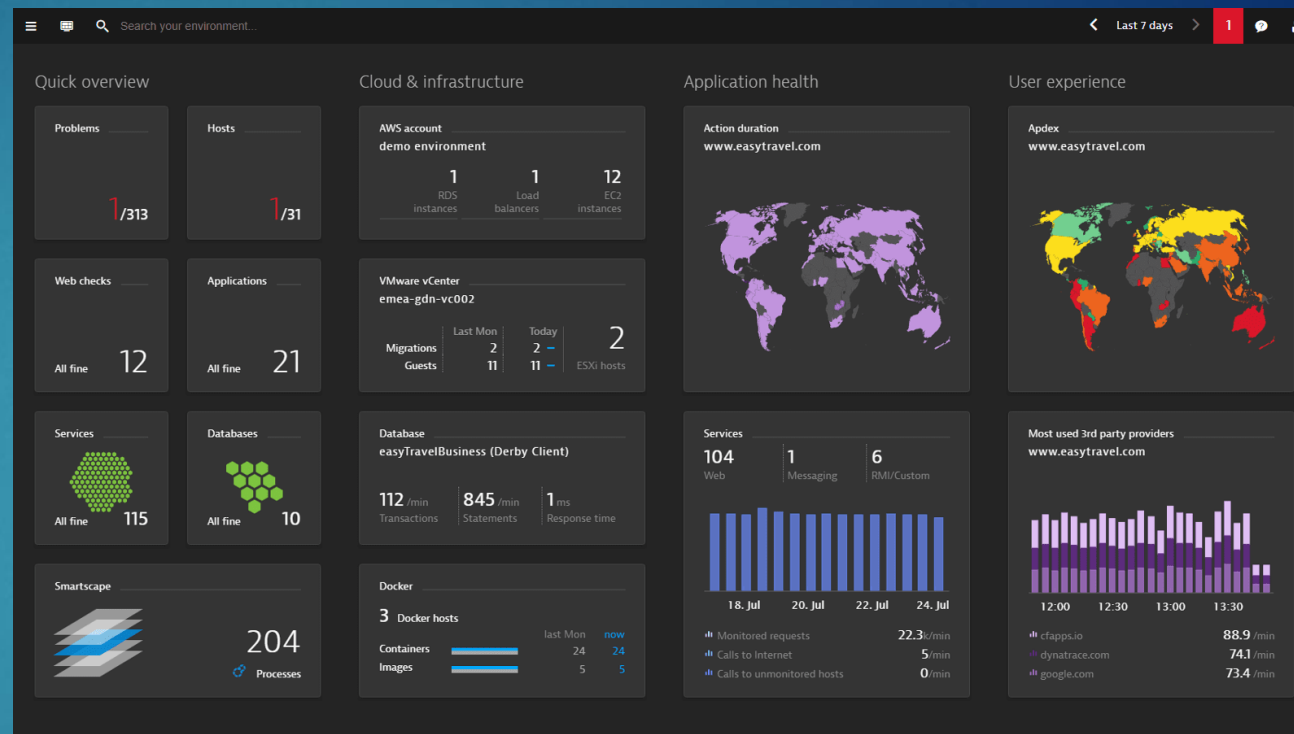
- ▶ Application monitoring that provides all performance metrics in real time and detects and diagnoses problems automatically
- ▶ 3 main components:
 - ▶ OneAgent – a single agent that automatically discovers, instruments, and collects high-fidelity monitoring data from everything in your IT environment
 - ▶ Smartscape – an interactive environment topology map that visualizes the dynamic relationships among all your application components across every tier
 - ▶ Davis AI engine – analyzes everything and tells you when there is a problem, the business impact of the problem, and the root cause of the problem so that you can fix it quickly



DynaTrace – Synthetic Monitoring

22

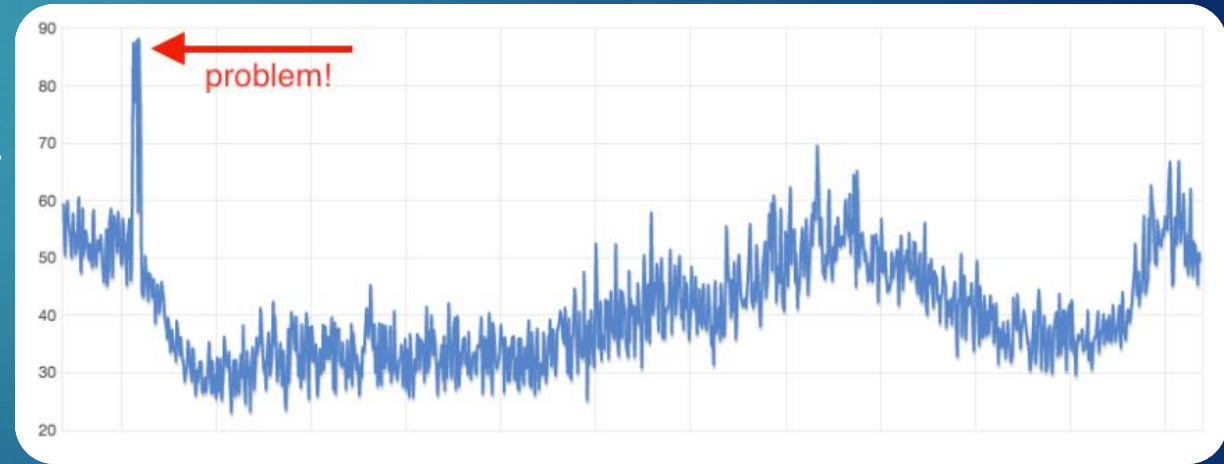
- ▶ Runs every 5 minutes
- ▶ Person A sends \$1 to Person B
- ▶ 5 minutes later Person B sends \$1 to Person A
- ▶ If this process ever fails, automatically contact support
- ▶ This process has allowed us to quickly react to issues much quicker before having significant customer impact



Monitoring and Metrics in the Cloud – CloudWatch

23

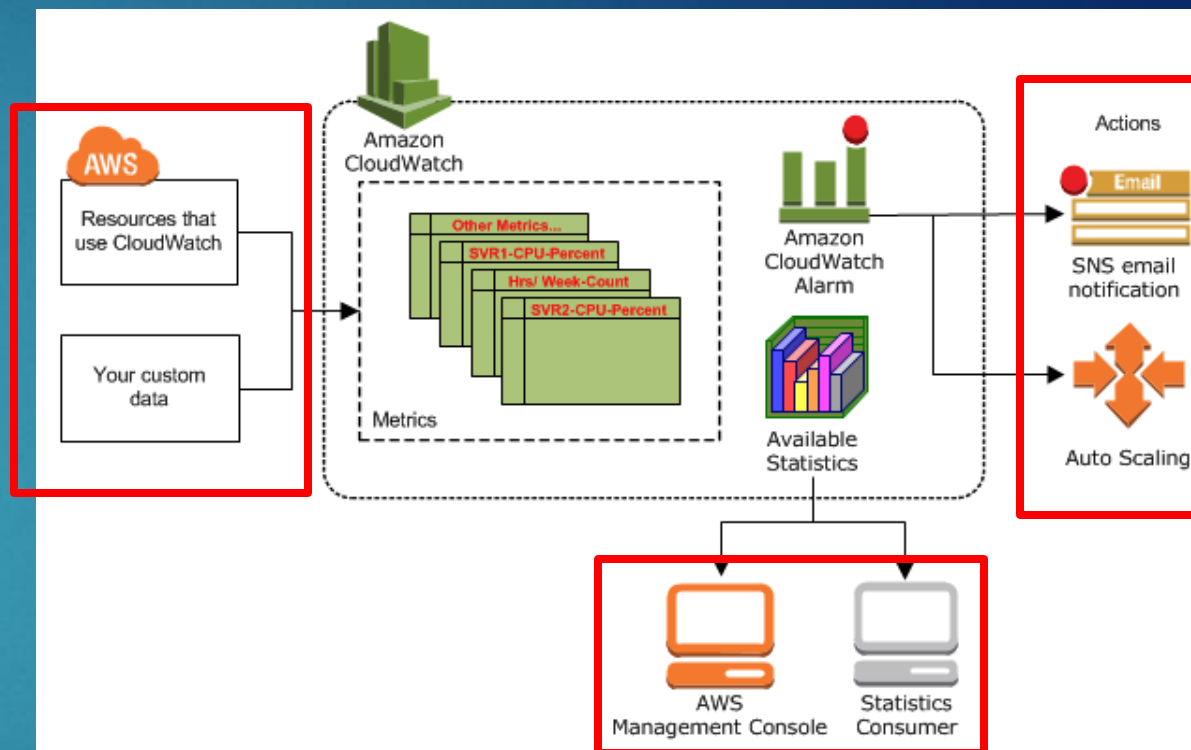
- ▶ CloudWatch is a service for monitoring AWS resources and the applications you run on AWS
- ▶ It allows developers, system architects, and administrators to monitor their AWS applications in the cloud, in near-real-time
- ▶ It is automatically configured to provide metrics on request counts, latency, and CPU usage
- ▶ You can use CloudWatch to collect and track metrics, collect and monitor log files, set alarms and automatically react to changes in AWS resources



CloudWatch – How it works

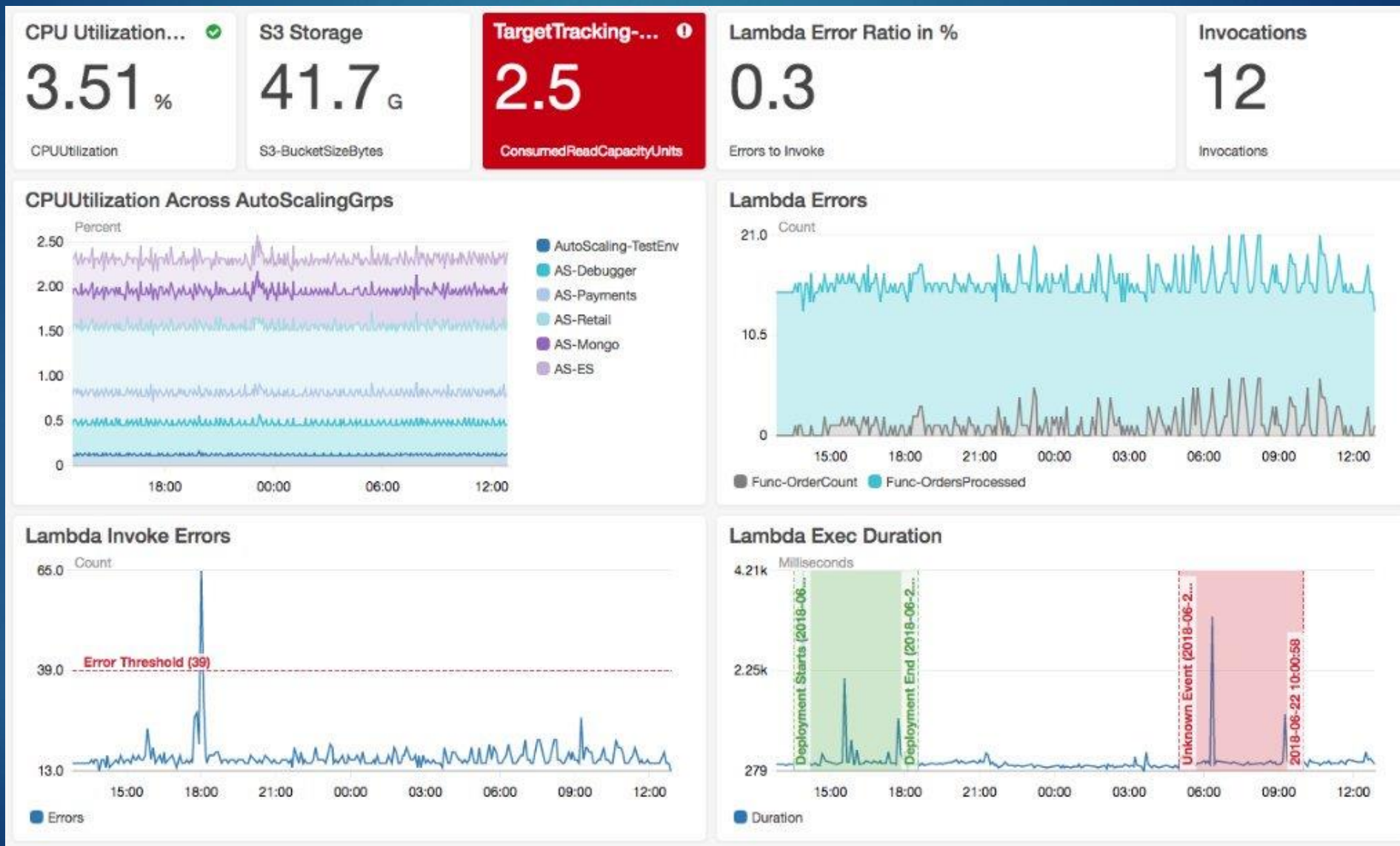
24

- ▶ CloudWatch is basically a metrics repository
- ▶ An AWS service (e.g. EC2) puts metrics into the repository, and you retrieve statistics based on those metric
- ▶ You can use metrics to calculate statistics and then present the data graphically in the CloudWatch console (e.g. Dashboard)
- ▶ You can configure alarm actions to stop, start, or terminate an Amazon EC2 instance when certain criteria are met
- ▶ In addition, you can create alarms that initiate Amazon EC2 Auto-Scaling and Amazon Simple Notification Service (Amazon SNS) actions on your behalf



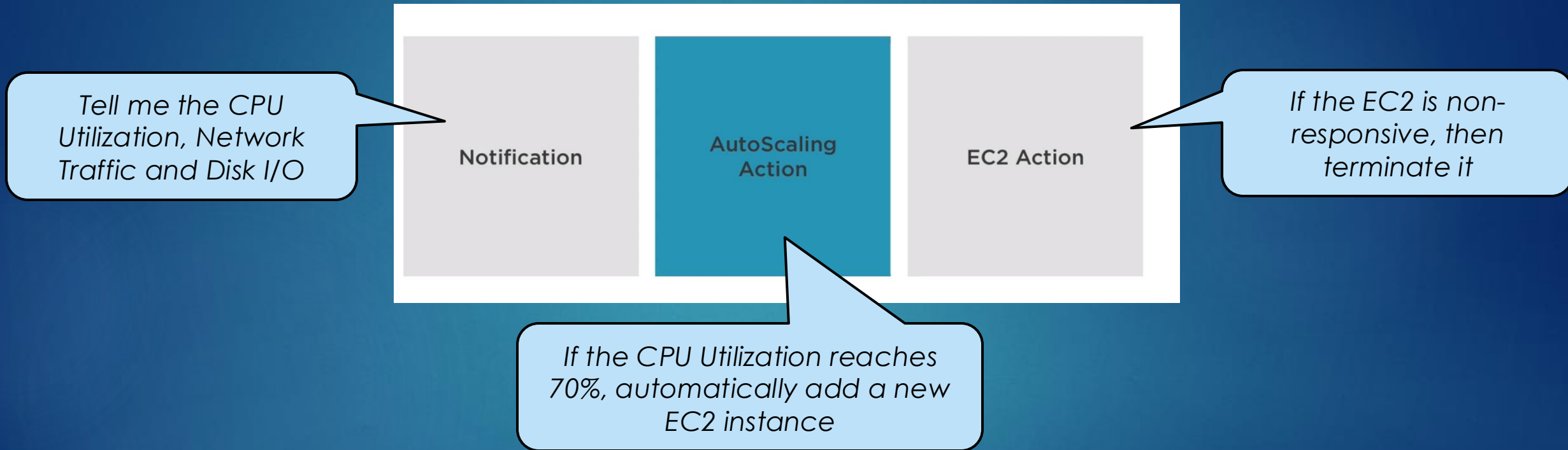
CloudWatch – Dashboard Example

25



CloudWatch – Alarm Actions Example

- Assume we have set up an alarm for an EC2 Instance...



- Actions can be combined to provide automated resiliency and scalability capabilities
- Example: "If the EC2 instance reports a status of `StatusCheckFailed_System`, reboot the EC2 and send a notification to the support team"

Advantages of CloudWatch

- ▶ One dashboard, access all data
 - ▶ Applications produce a lot data as they are highly distributed
 - ▶ Access all the data which have been collected can be displayed on a single CloudWatch dashboard
- ▶ Improve total cost of ownership
 - ▶ CloudWatch can be used to set alarms and can take automated actions while there is a breach in the limits provided, which can help to minimize the costs spent on AWS services
- ▶ Insights from logs
 - ▶ You receive detailed insights on separate AWS services and the applications you run on the infrastructure.
 - ▶ Data like memory, CPU utilization, and capacity utilization can be monitored and receive insights from it
- ▶ Optimize Applications and resources
 - ▶ Using the log and metric data, you can optimize your AWS services to provide maximum throughput and performance



Amazon
CloudWatch

CloudWatch Limitations

- ▶ Limited data retention, which is restricted to 2 weeks of metrics data
- ▶ CloudWatch can't monitor non-AWS cloud services or user-managed infrastructure
- ▶ Graphs and text-based widgets are limited in functionality
- ▶ Basic monitoring collects data at a frequency of 5 minutes, which might not be frequent enough for managing scale in a critical environment
 - ▶ Detailed monitor comes at an additional cost, which can add up quickly
- ▶ For larger organizations, it might make more sense to use it in conjunction industry standard monitoring tools



Cloud Monitoring Solutions

29

- ▶ To recap, CloudWatch focuses on information for health and performance monitoring
- ▶ There are other 3rd-party solutions that can combine CloudWatch data to provide more wholistic reporting



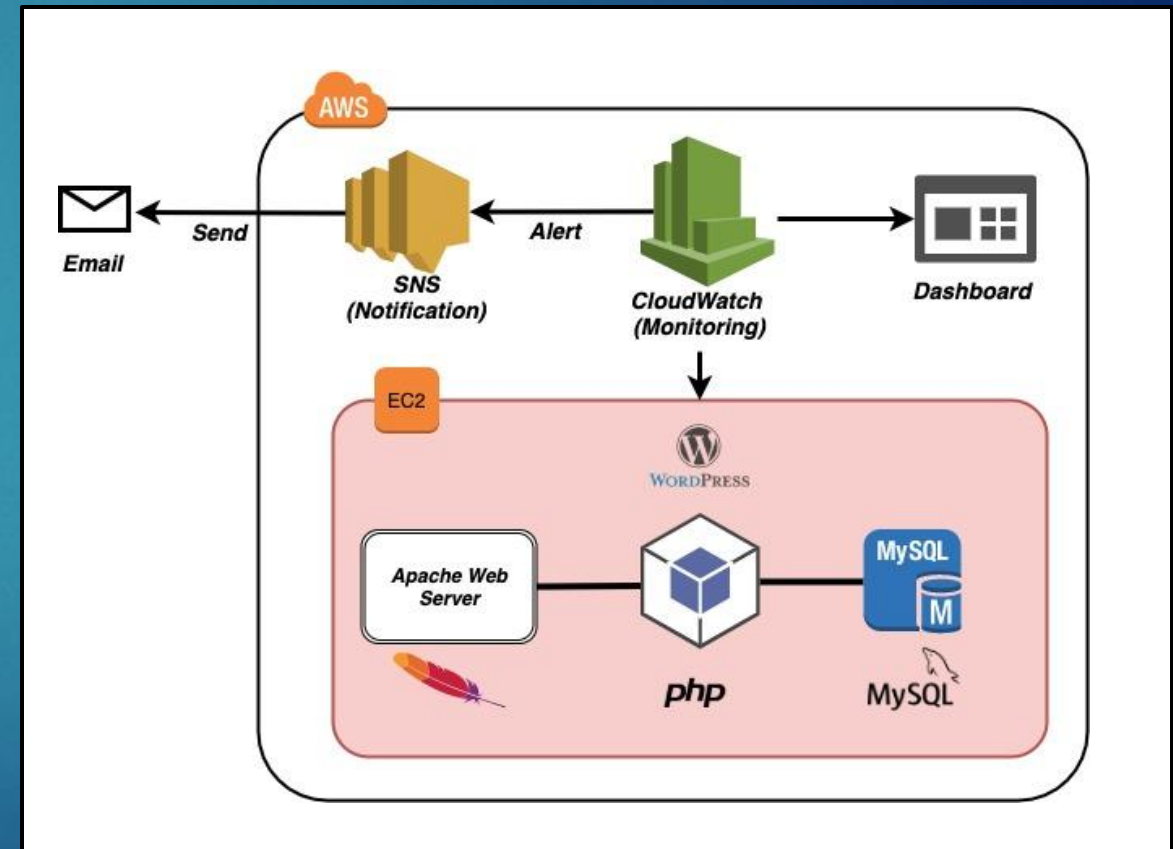
Datadog + CloudWatch

30



CloudWatch – Last Activity

- ▶ You will create a simple alarm and dashboard to monitor an application we've ran before (WordPress)
- ▶ The dashboard will monitor a basic metric like CPU usage
- ▶ You will be doing a simple “chaos engineering” task of overwhelming the CPU to trigger an alarm
- ▶ Located at: **Assignments > Activity– Create a CloudWatch Dashboard and Alarm**

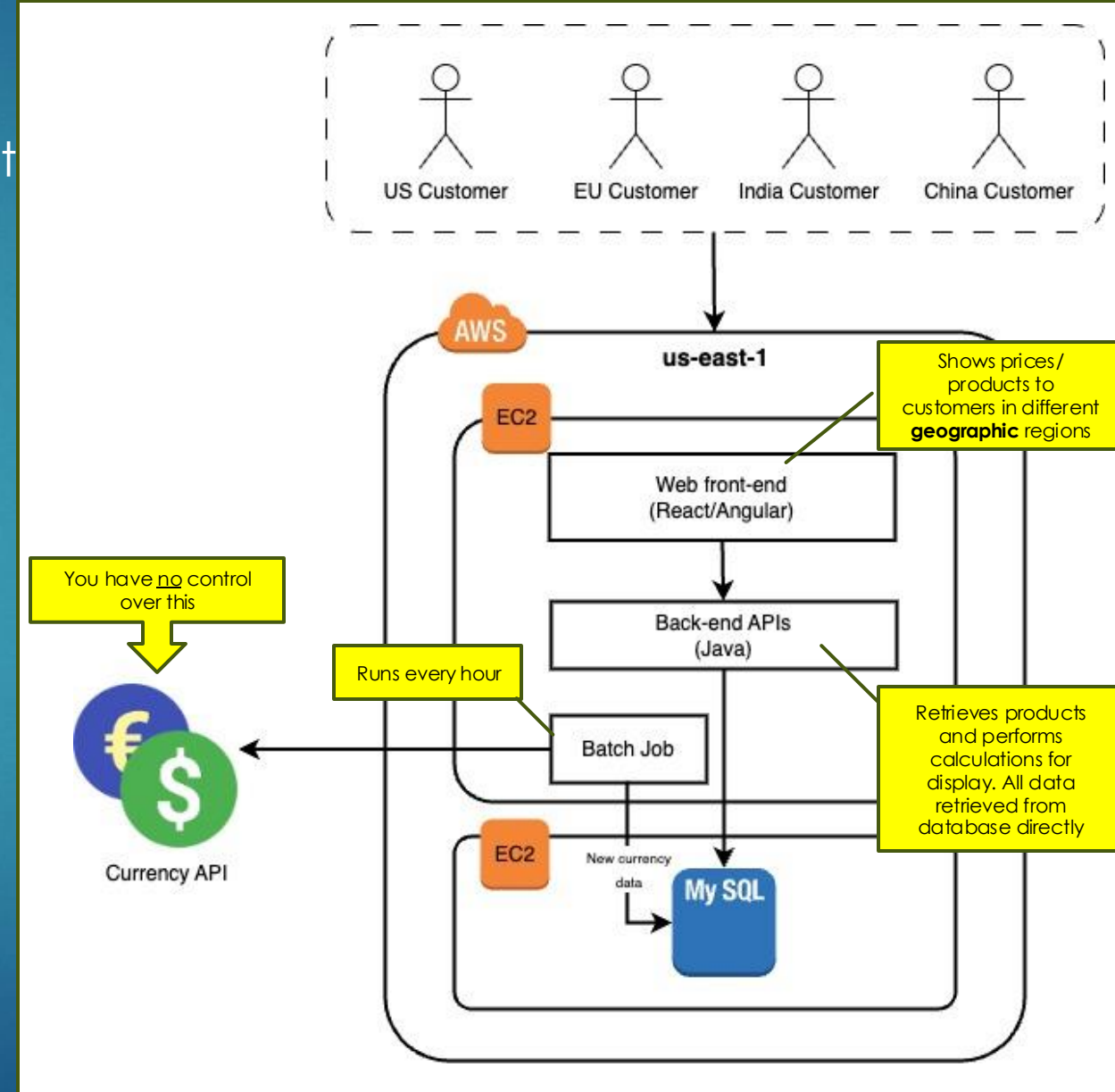


Last Mistake to Avoid: Do You Really Need to Build it?

- ▶ It can be hard for development teams to accept the fact that maybe buying might be a better idea vs. building
- ▶ For commodity-based solutions, look at SaaS and objectively compare the costs
- ▶ It might be more costly upfront (e.g. licensing) for a “buy” solution, but long term it may fair outweigh the support headaches with a “build”
- ▶ Avoid “Not invented here syndrome” (NIHS)
 - ▶ This is the name for the tendency of both individual developers and entire organizations to reject suitable external solutions to software development problems in favor of internally developed solutions
 - ▶ NIHS can be defined as a situation where an external solution is rejected only because it was not internally developed - in other words, there are no other factors that dictate an internally developed solution would be superior



- ▶ For your final discussion topic, you've been presented with the following architecture that needs to run on AWS
- ▶ Some requirements:
 - ▶ Needs to scale with web traffic (e.g. Black Friday)
 - ▶ Downtime = lost revenue
 - ▶ Jobs management must be low maintenance
 - ▶ Business barely has a development staff
 - ▶ All applications layers need to be monitored
 - ▶ Needs to be performant for global/regional users
 - ▶ Costs need to be minimized
- ▶ Each team has 10 minutes to come up with their recommendations for a better solution
- ▶ Template and diagram can be found here:
 - ▶ [Assignments > Activity - AWS Architecture Recommendations](#)



Example Solution

- ▶ Use of AWS Cloudfront (CDN)/S3 for web site hosting
- ▶ Implement caching to minimize database access
- ▶ Amazon RDS is a managed DB, minimal overhead. The only server 'running' at any given moment
- ▶ AWS Lambda gives us on-demand code execution with no server maintenance
- ▶ Cloud Monitoring (e.g Datadog) to detect and alert for any degradations in performance

